DevSecOps Case Study Report

Repository: `github.com/luonglt20/AegisFlow`
Target dùng chung cho Task 1-4: OWASP Juice Shop
Môi trường chạy: local bằng `docker-compose.yml`
Pipeline dùng chung: `pipeline/run_pipeline.sh`

1. Phạm vi và cách đáp ứng yêu cầu chung

Yêu cầu	Cách đáp ứng trong repo
Một repository duy nhất cho toàn bộ Practical Exercise	Repo này chứa dashboard, server, pipeline, tài liệu, target app mẫu và artifact scan.
Các task dùng cùng một ứng dụng	Task 1-4 cùng dùng OWASP Juice Shop.
Các task dùng cùng một CI/CD pipeline	Task 1-4 cùng đi qua `pipeline/run_pipeline.sh` và cùng được điều phối bởi `server.py`.
Các task chạy trong cùng một môi trường local bằng Docker / docker-compose	Repo dùng `docker-compose.yml` để dựng dịch vụ AegisFlow và target app trong cùng môi trường.
Hoàn thành tối thiểu 03 task	Repo này thực hiện Task 1, Task 2, Task 3 và Task 4.

2. Hướng dẫn chạy chung

1. Chạy `docker compose up --build` tại root repository.
2. Mở dashboard tại `http://localhost:58081`.
3. Chọn target `juice-shop` để dùng chung cho Task 1-4.
4. Nhấn chạy scan để kích hoạt pipeline.
5. Theo dõi trạng thái trên dashboard và kiểm tra artifact trong `security-results/`.

3. Kiến trúc ngắn gọn

```
Dashboard -> server.py -> run_pipeline.sh
-> build_target.py
-> test_target.py
-> scan_sast.py
-> scan_sca.py
-> scan_sbom.py
-> scan_secret.py
-> scan_iac.py
-> scan_dast.py
-> report_generator.py
-> ai_triage_engine.py
-> policy_engine.py
-> audit_logger.py
-> security-results/*.json
-> dashboard/data/*.json
```

API
`server.py`
Nhận request scan và đồng bộ artifact

Pipeline
`run_pipeline.sh`
Điều phối toàn bộ build, test, security, report

Storage
`security-results/`
Nơi lưu report dùng cho dashboard và policy

4. Task 1 - CI/CD Pipeline + SAST

4.1 Yêu cầu kỹ thuật

Yêu cầu	Cách triển khai
Pipeline tối thiểu gồm Build, Test, Security	Được thực hiện bởi `build_target.py`, `test_target.py` và nhóm script `scan_*.py`.
Optional deploy mock	Không triển khai deploy mock; phạm vi repo tập trung vào scan, policy và reporting.
Tích hợp SAST scan	Dùng Semgrep qua `scan_sast.py`.
Sinh report scan	Xuất report dưới dạng SARIF/JSON trong `security-results/`.
Pipeline parse report để ra pass/fail	`report_generator.py` hợp nhất finding; `policy_engine.py` dùng report này để ra quyết định.

4.2 Phân tích

Câu hỏi	Trả lời
Vì sao chọn SAST tool này?	Chọn Semgrep vì nhẹ, dễ chạy trong container, không cần server riêng và dễ xuất SARIF/JSON để parse.
SAST chạy ở stage nào? Vì sao?	Chạy sau Build/Test và trước các bước scan khác để phát hiện lỗi source code sớm, giảm chi phí sửa lỗi.
Issue nào block, issue nào warning?	Critical block ngay; từ 2 High trở lên block; Medium warning; Low allow và đưa vào backlog.

4.3 Hướng dẫn chạy demo

1. Chạy toàn bộ môi trường bằng `docker compose up --build`.
2. Mở dashboard và chọn target `juice-shop`.
3. Chạy scan và chờ pipeline hoàn tất.
4. Mở `security-results/sast_results.sarif` hoặc report tương ứng để xem finding SAST.
5. Kiểm tra quyết định pass/fail trong `policy_result.json` và trên dashboard.

4.4 Output

Artifact	Đường dẫn
Pipeline config local	`pipeline/run_pipeline.sh`
Pipeline config CI tham chiếu	`.gitlab-ci.yml`
SAST report mẫu	`security-results/sast_results.sarif`
Mô tả trong PDF	Phần 4 của tài liệu này

Lưu ý: nếu target không hỗ trợ build/test đầy đủ trong local demo, hệ thống có thể chạy validation mode. Giới hạn này được giữ rõ để phân biệt demo pipeline với production CI hoàn chỉnh.

5. Task 2 - SCA + Container / IaC Security

5.1 Yêu cầu kỹ thuật

Yêu cầu	Cách triển khai
SCA	Dùng Trivy qua `scan_sca.py`.
Container / filesystem scanning	Trivy quét target và thành phần liên quan trong local runtime.
IaC scanning	Dùng Checkov qua `scan_iac.py` cho Dockerfile / docker-compose / workflow.
SBOM	Dùng Syft qua `scan_sbom.py`.

5.2 Phân tích

Nội dung bắt buộc	Trả lời
Vulnerability do code	Lỗi nằm trong source code và phải sửa trong code.
Vulnerability do dependency	Lỗi nằm ở package, library hoặc base image và phải sửa ở dependency/base image.
Misconfiguration	Lỗi cấu hình nằm ở Dockerfile, docker-compose hoặc workflow và phải sửa ở config/pipeline.

Issue mẫu High/Critical	Root cause	Fix ở đâu
Base image cũ, ví dụ `node:14-alpine`	Base image EOL hoặc chứa CVE.	Sửa Dockerfile / base image.
Secret hoặc private key bị commit vào repo	Secret management chưa tách khỏi source control.	Sửa quy trình secret management, rotate secret, xóa khỏi repo history nếu cần.

5.3 Hướng dẫn chạy demo

1. Chạy môi trường bằng `docker compose up --build`.
2. Chạy scan từ dashboard.
3. Mở các file `sca_results.json`, `iac_results.json`, `sbom.json`.
4. Đối chiếu đề xuất remediation trong report hợp nhất.

5.4 Output

Artifact	Đường dẫn
SCA report	`security-results/sca_results.json`
IaC report	`security-results/iac_results.json`
SBOM	`security-results/sbom.json`
Remediation	`security-results/full_report.json` hoặc `full_report_triaged.json`

6. Task 3 - Policy Enforcement & Security Governance

6.1 Scenario bắt buộc

Scenario yêu cầu pipeline phát hiện: **1 Critical, 2 High, 2 Medium**.

Với policy hiện tại của AegisFlow, scenario này sẽ bị **BLOCKED** vì có Critical và cũng vượt ngưỡng High.

6.2 Thiết kế policy enforcement

```
IF critical_count > 0
  THEN BLOCK
ELSE IF high_count >= 2
  THEN BLOCK
ELSE IF risk_weight > 50
  THEN WARNING
ELSE
  PASS
```

Nội dung	Thiết kế
Điều kiện block / allow	Critical block; từ 2 High trở lên block; Medium/Low theo warning hoặc allow.
Exception có cho phép không?	Có, nhưng phải có owner, thời hạn và lý do.
Ai approve?	AppSec Lead + Engineering Manager; có thể cần Product Owner tùy mức ảnh hưởng release.

6.3 Dashboard / reporting

Nhóm người xem	Thông tin cần nhìn thấy
CISO / BOD	Security score, số lượng Critical/High, trạng thái pass/fail, audit trail.
Dev team	Finding cụ thể, file/line, root cause, remediation.
Product	Ảnh hưởng tới release, SLA và exception status.

6.4 Quy trình xử lý

Câu hỏi	Trả lời
Ai chịu trách nhiệm fix?	Dev owner là người sửa chính; AppSec hỗ trợ triage và xác nhận risk.
SLA cho Critical / High?	Critical: 24 giờ. High: 72 giờ.
Giao tiếp với Product như thế nào?	Thông báo tác động tới release, xác nhận deadline remediation hoặc risk acceptance.

6.5 Hướng dẫn chạy demo

1. Chạy pipeline trên cùng target đã dùng cho Task 1-2.
2. Mở `policy_result.json` sau khi scan hoàn tất.
3. Đối chiếu severity summary với logic policy bên trên.
4. Mô tả trên dashboard nhóm thông tin dành cho CISO/BOD, Dev team và Product.

6.6 Output

Artifact	Đường dẫn
Sơ đồ logic policy	Phần 6.2 của tài liệu này
Policy result	`security-results/policy_result.json`
Dashboard mock / reporting	`dashboard/` và `dashboard/data/`
Step-by-step process	Phần 6.5 của tài liệu này

7. Task 4 - Security of the Pipeline

7.1 Yêu cầu

Phân tích bản thân pipeline như một hệ thống cần được bảo vệ và liên hệ trực tiếp với pipeline đã build ở Task 1-2.

7.2 Threat -> Impact -> Mitigation

Threat	Impact	Mitigation trong pipeline
Secrets leakage	Lộ API key hoặc token.	Gitleaks, env var, không commit secret vào repo.
Compromised CI runner	Sửa artifact hoặc đánh cắp biến môi trường.	Runner container hóa, giảm privilege, audit log.
Malicious pipeline modification	Bypass scanner hoặc quality gate.	Code review cho `pipeline/` và CI config.
Over-permissioned service account	Token có quyền quá rộng.	Least privilege, token read-only mặc định.
Supply chain attack	Tool hoặc dependency build bị thay thế độc hại.	Pin version scanner, SBOM, quét dependency.

7.3 Demo minh họa

1. Dùng cùng pipeline ở Task 1-2 để chạy secret scan, SCA, IaC scan và policy.
2. Kiểm tra artifact trong `security-results/` để chỉ ra rủi ro đã được phát hiện ở đâu.
3. Đối chiếu mitigation tương ứng trong Dockerfile, compose hoặc pipeline scripts.

7.4 Output

Artifact	Đường dẫn
Bảng threat -> mitigation	Phần 7.2 của tài liệu này
Secret scan report	`security-results/secret_scan_results.json`
IaC / dependency evidence	`security-results/iac_results.json`, `sca_results.json`, `sbom.json`

8. Kết luận

Repo AegisFlow đáp ứng đủ yêu cầu practical exercise cho Task 1-4 trên cùng một ứng dụng, cùng một pipeline và cùng một môi trường Docker Compose. Tài liệu này đóng vai trò vừa là hướng dẫn chạy lại, vừa là báo cáo kỹ thuật theo đúng các mục yêu cầu của đề bài.